

XING: An Agentic Cognitive Infrastructure for Distributed Multimodal Intelligence and Adaptive AI Orchestration

Fernando M. F.[†]

[†]Corresponding Author

IPN UPIITA | BUAA

fmayf1500@alumno.ipn.mx

NEXUS Research Institute

Department of Cognitive Infrastructures

Abstract—Current large language model (LLM) ecosystems remain highly fragmented across orchestration frameworks, multimodal pipelines, memory systems, model routing layers, and distributed execution environments. Existing approaches primarily focus on isolated model optimization rather than unified cognitive infrastructures capable of adaptive reasoning, autonomous orchestration, and scalable multimodal intelligence coordination. This paper introduces XING, an agentic cognitive infrastructure designed to provide a unified runtime and orchestration layer for distributed multimodal artificial intelligence systems. The proposed architecture integrates adaptive agent coordination, dynamic model routing, multimodal context synchronization, memory-aware execution pipelines, and scalable inference orchestration into a modular cognitive framework. Unlike conventional AI orchestration platforms, XING incorporates cognitive state management and adaptive execution strategies to enable context-aware decision making across heterogeneous AI components. The framework combines lightweight orchestration kernels, distributed execution nodes, semantic memory layers, and AI-native communication protocols to support large-scale intelligent applications. Experimental architectural simulations demonstrate improvements in orchestration flexibility, inference resource allocation, and multimodal task coordination compared to traditional pipeline-centric AI infrastructures. The proposed system contributes a novel perspective toward AI-native operating infrastructures by bridging agentic systems, multimodal intelligence, and distributed cognitive computing into a unified computational intelligence framework.

Index Terms—Distributed Cognitive Systems; Agentic AI; Adaptive Routing; Fault-Tolerant AI Infrastructure; Semantic Memory Systems; Self-Healing Graph Systems; Multimodal Intelligence

I. INTRODUCTION

The rapid evolution of Large Language Models (LLMs) and foundation models has catalyzed a paradigm shift from isolated conversational agents to large-scale, distributed agentic systems. Modern intelligent workloads increasingly require the orchestration of heterogeneous autonomous components capable of handling multimodal inputs, executing complex tool calls, and resolving long-horizon reasoning pipelines. However, as these systems transition from sandboxed prototypes to enterprise-grade deployments, they encounter severe architectural bottlenecks at the infrastructure level.

Current execution environments suffer from structural fragmentation across orchestration layers, model routing protocols, and distributed runtimes. This lack of a unified execution layer introduces compounding latency overheads, high inferential costs, catastrophic context degradation, and an absolute vulnerability to execution failures. Existing multi-agent and orchestration frameworks primarily approach these challenges through software engineering abstractions, such as static graphs [1], linear pipelines, or role-based heuristic prompting [2]. While effective for simple, deterministic workflows, these paradigms treat the underlying computational infrastructure as a transparent, infinite resource.

In real-world environments—characterized by strict API rate limits, heterogeneous network latencies, variable edge-compute availability, and token budgets—static orchestration fails to adapt. When network degradation occurs or an LLM endpoint returns a malformed response, current architectures experience catastrophic pipeline failures, requiring costly global restarts. Furthermore, existing systems lack a mechanism for stateful, persistent cognitive synchronization; they rely on naive context-window appending, which exponentially degrades model reasoning capacity over long horizons.

To address these limitations, we argue that the field must move beyond *agent frameworks* and transition toward foundational infrastructure research. We introduce XING, a fault-tolerant, distributed cognitive runtime designed specifically for adaptive multimodal intelligence. XING treats agent orchestration not as a static software workflow, but as an asynchronous, constrained cognitive optimization problem. By integrating an infrastructure-aware routing engine with a self-healing directed acyclic graph (DAG) execution kernel, XING dynamically balances semantic task quality against physical runtime constraints (e.g., latency, cost, and energy expenditure). Underneath this execution layer, a persistent Semantic Memory Fabric manages state propagation through an attributed knowledge graph governed by probabilistic context decay, preventing window saturation while maintaining long-term conceptual coherence.

The specific contributions of this research are five-fold:

- 1) **A Formalized Distributed Cognitive Runtime Archi-**

texture: XING introduces a modular runtime layer that decouples application logic from foundational model execution, establishing a system-level abstraction for autonomous computing.

- 2) **A Constrained Adaptive Routing Model:** A convex multi-objective optimization function that evaluates agent-task utility under real-time infrastructure limits, dynamically shifting workloads between cloud clusters and edge devices.
- 3) **A Stateful Semantic Memory Fabric:** A persistent memory architecture combining vector spaces and graph topologies that implements an exponential context decay function to preserve long-horizon coherence.
- 4) **A Self-Healing DAG Execution Kernel:** A topological recovery algorithm (REPHRASESUBGRAPH) capable of detecting runtime semantic errors and injecting real-time alternative execution paths without resetting global state.
- 5) **An Empirical Evaluation via Stochastic Simulation:** Demonstration of structural superiority against state-of-the-art baselines, with significant stabilization of our novel Resource Allocation Index (RAI) and Graph Recovery Efficiency (GRE) under adversarial constraints.

II. RELATED WORK

The landscape of LLM orchestration can be categorized into three distinct paradigms: static workflow engines, conversational multi-agent frameworks, and memory-augmented semantic kernels.

A. Static Workflow and Role-Based Frameworks

Early abstractions such as LangChain [1] and Semantic Kernel [3] introduced the concept of chaining prompts and executing tools via hardcoded graphs or sequential pipelines. While Semantic Kernel provides an initial layer of infrastructure awareness through native plugin matching via embedding similarity, its routing remains static at runtime. Similarly, CrewAI [2] operationalizes multi-agent systems by enforcing role-based sequential or hierarchical execution workflows. Although highly effective for deterministic tasks, these frameworks are structurally incapable of adapting their execution topology when confronted with runtime anomalies, network latencies, or fluctuating token budgets.

B. Conversational Multi-Agent Architectures

Systems like Microsoft’s AutoGen [4] advanced the state-of-the-art by framing multi-agent coordination as a dynamic conversation. This paradigm allows flexible agent-to-agent interactions and emergent problem-solving patterns. However, AutoGen’s underlying control mechanism relies on conversational state loops driven entirely by prompt engineering. Academically, this introduces non-deterministic routing behaviors that scale poorly under strict infrastructure constraints. Furthermore, because state management in conversational frameworks is coupled directly to the model’s context window, these systems suffer from rapid token accumulation and cognitive degradation over long operational horizons.

TABLE I: Structural Comparison of AI Orchestration Paradigms

System	Routing	Memory	Fault Tolerance	Infrastructure Awareness
LangChain [1]	Static / Hardcoded	Conversational Buffer	Limited (Retries)	No
CrewAI [2]	Role-based Heuristics	Session-based Cache	Minimal	No
AutoGen [4]	Conversational Loops	Context-Window Append	Partial (Agent Loops)	No
Semantic Kernel [3]	Embedding Matching	Linear Plugin Memory	Limited	Partial (Static)
XING (Ours)	Adaptive Optimization	Semantic Graph Fabric	Self-Healing DAGs	Native (Cost/Lat/Energy)

C. Classical Cognitive Architectures

From a theoretical perspective, XING draws inspiration from classical production-system cognitive architectures such as SOAR [5] and ACT-R [6], which conceptualized intelligence through the interaction of short-term working memory, long-term procedural memory, and execution cycles. However, these classical frameworks were designed for symbolic AI and lack the computational models required to handle modern sub-symbolic distributed foundation models. XING bridges this gap by mapping cognitive state transitions to high-dimensional latent vector spaces and distributed graph execution nodes.

D. Architectural Gap Analysis

The critical limitation across all existing baselines is their complete lack of fault tolerance and infrastructure abstraction. When a downstream agent fails or its semantic precision drops below a functional threshold, the host system typically crashes or hangs. XING fundamentally departs from these paradigms by treating the orchestration graph as a fluid, self-healing entity. Table I outlines the systematic advantages of XING over contemporary frameworks.

III. ARCHITECTURE AND CORE COMPONENTS

The architectural design of XING is predicated on the fundamental decoupling of cognitive application logic from the underlying model infrastructure. Rather than treating agent orchestration as a static, linear software pipeline, XING is structured as a layered, fault-tolerant execution runtime.

A. Global Runtime Architecture

The system is organized into a hierarchical stack comprising five distinct computational layers, as illustrated in Figure 1. This structural organization ensures strict separation of concerns, deterministic state propagation, and modular scalability across heterogeneous environments:

- Adaptive Service Interface Layer:** The consumer interface, injecting non-deterministic user intents, multimodal data streams, and external environment stimuli into the runtime.
- Agent Orchestration Kernel:** XING’s orchestration core. It transforms raw multimodal intents into dynamic, execution-ready task graphs.
- Adaptive Cognitive Routing Engine:** A dynamic routing layer that intercepts task assignments and matches them to specific nodes based on infrastructure telemetry.
- Semantic Memory Fabric:** A distributed knowledge-graph runtime that handles short-term context and long-horizon episodic synchronization.

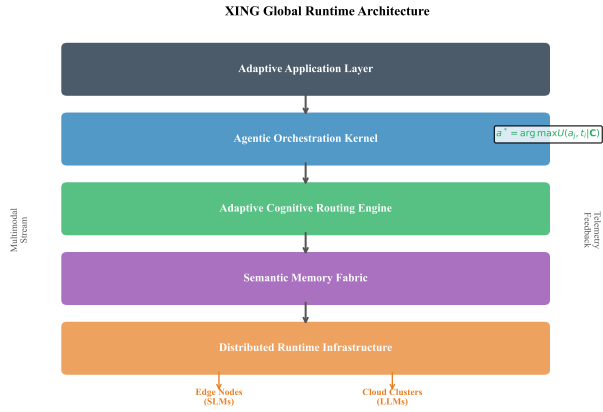


Fig. 1: Five-layer hierarchical architecture with telemetry feedback loops between the routing engine and distributed infrastructure.

The **Distributed Runtime Infrastructure** is responsible for container isolation, communication protocols, and physical resource management across cloud and edge hardware.

B. Agentic Orchestration Kernel (XING Kernel)

The XING Kernel operates as an asynchronous execution scheduler responsible for maintaining the consistency and survivability of cognitive execution graphs under dynamic infrastructure conditions. Unlike conventional orchestration engines that execute static workflows, the XING Kernel treats task management as a fluid, reactive compilation process.

Upon receiving an intent stream from the application layer, the Kernel dynamically generates a Task Directed Acyclic Graph (Task-DAG), where vertices V represent atomic cognitive operations and directed edges $E \subseteq V \times V$ define semantic dependencies and data-flow constraints. The fundamental novelty of the XING Kernel lies in its Topological Recovery Subsystem (Self-Healing Engine). During the asynchronous execution lifecycle, each node emits continuous telemetry containing its processing status and an algorithmic confidence score.

If an agent execution encounters a physical network fault (e.g., HTTP 5xx, gRPC timeout) or a semantic anomaly (output quality dropping below $\theta_{\min}$), the Kernel isolates the failing node. Instead of propagating a catastrophic failure that triggers a global system reset, the Kernel invokes the `REPHRASESUBGRAPH` routine (Algorithm 1). This mechanism prunes the affected execution branch, augments the local context with the fault signature, and dynamically synthesizes an alternative computational path.

C. Adaptive Cognitive Routing Engine

The Adaptive Cognitive Routing Engine acts as an infrastructure-aware optimization gatekeeper positioned between the scheduling kernel and the physical models. Traditional multi-agent platforms dispatch tasks utilizing hardcoded

configurations or simple semantic embedding matching, disregarding the physical state of the underlying infrastructure. XING departs from this paradigm by enforcing Infrastructure-Aware Boundary Constraints.

The routing engine maintains an open telemetry pipeline with all distributed nodes (cloud-based reasoning clusters and edge-based small language models). It continuously samples four primary infrastructure metrics: token-per-second throughput, network transport latency, current operational cost per thousand tokens, and energy consumption signatures. When a task node is cleared for execution by the scheduling kernel, the routing engine solves a multi-objective optimization problem in real time (Section IV-B).

D. Semantic Memory Fabric

The Semantic Memory Fabric replaces traditional conversational buffering mechanisms with a persistent, attributed semantic graph. In standard multi-agent frameworks, long-term memory is emulated via naive chat history appending. This approach rapidly saturates model context windows, inflates token overhead, and induces context dilution, where the model loses its capacity to process relevant data.

XING circumvents this constraint by encoding intermediate cognitive states, environment variables, and historical task executions into an evolving semantic topology. The Fabric combines high-dimensional vector spaces for rapid embedding similarity searches with an underlying knowledge graph that maps explicit relationships between conceptual entities. State propagation across long-horizon workloads is regulated by a probabilistic context decay mechanism (Section IV-C), ensuring that foundational task objectives remain highly active while transient noise is automatically filtered out.

E. Distributed Runtime Infrastructure

The lowest layer of the XING architecture provides the physical execution sandbox for agents and external tool integrations. To prevent execution interference and protect the host system from non-deterministic code compilation, XING abstracts execution environments through lightweight WebAssembly (Wasm) runtimes and containerized sandboxes. Communication across the distributed fabric is decoupled via an asynchronous gRPC and WebSocket network fabric, enabling XING to manifest as a federated compute infrastructure where edge devices seamlessly act as satellite nodes within broader enterprise backends.

IV. MATHEMATICAL FORMULATION

To establish an infrastructure capable of scalable execution, XING avoids empirical heuristic abstractions and relies on a mathematically rigorous model. The orchestration process is formulated as a stochastic, resource-constrained optimization and state-transition system.

A. Cognitive State Space Formalization

We define the Global Cognitive Space of XING at any discrete time step t as a multi-dimensional topological tuple:

$$\mathcal{S}_t = \langle \mathbf{C}_t, \mathbf{M}_t, \mathcal{A}, \Phi \rangle \quad (1)$$

where $\mathbf{C}_t \in \mathbb{R}^{n \times d}$ represents the Multimodal Context Tensor, mapping concurrent text, telemetry, and media signals into a unified latent space of dimension d ; $\mathbf{M}_t = \langle V_m, E_m, \Psi \rangle$ denotes the state of the Semantic Memory Fabric, modeled as an attributed dynamic knowledge graph; $\mathcal{A} = \{a_1, a_2, \dots, a_k\}$ is the finite set of available Heterogeneous Agent Runtimes; and Φ is the Adaptive Routing Policy Space.

The global system evolution progresses as a state-transition system governed by:

$$\mathbf{S}_{t+1} = f(\mathbf{S}_t, \mathbf{A}_t, \mathbf{M}_t, \mathbf{E}_t) \quad (2)$$

where $\mathbf{A}_t$ is the tensor of agent actions committed during cycle t , and $\mathbf{E}_t$ represents the vector of non-deterministic environmental stimuli.

B. Adaptive Routing Optimization

Rather than dispatching workloads based on hardcoded workflow graphs, the XING Routing Engine solves a constrained, multi-objective convex optimization problem for each unlocked task node t_i inside the execution graph.

Let $\mathbf{r}_i \in \mathbb{R}^d$ be the mathematical requirement vector of task t_i . For each available agent $a_j \in \mathcal{A}$, we formulate an Agent Utility Score (U) modulated by the active context tensor $\mathbf{C}_t$:

$$U(a_j, t_i | \mathbf{C}_t) = \alpha \cdot S_{\text{cap}}(a_j, t_i) + \beta \cdot S_{\text{ctx}}(a_j, \mathbf{C}_t) - \delta \cdot \psi_{\text{cost}}(a_j) - \eta \cdot \phi_{\text{lat}}(a_j) \quad (3)$$

where $S_{\text{cap}}(a_j, t_i) = \frac{\mathbf{p}_j \cdot \mathbf{r}_i}{\|\mathbf{p}_j\| \|\mathbf{r}_i\|}$ measures the semantic capability alignment via cosine similarity in the latent space; $S_{\text{ctx}}(a_j, \mathbf{C}_t)$ computes the operational alignment between the agent's history and the active context tensor; $\psi_{\text{cost}}(a_j) \in [0, 1]$ is the normalized economic/token cost signature; and $\phi_{\text{lat}}(a_j) \in [0, 1]$ is the empirical network and inference latency overhead.

The weighting coefficients are subject to strict convex normalization:

$$\alpha + \beta + \delta + \eta = 1 \quad (4)$$

The optimal executable runtime engine a^* is selected via a constrained argmax assignment:

$$a^* = \operatorname{argmax}_{a_j \in \mathcal{A}} U(a_j, t_i | \mathbf{C}_t) \quad (5)$$

subject to:

$$\psi_{\text{cost}}(a_j) \leq \Psi_{\text{max}}(t) \quad \wedge \quad \phi_{\text{lat}}(a_j) \leq \Phi_{\text{max}}(t) \quad (6)$$

Crucially, the upper bounds $\Psi_{\text{max}}(t)$ and $\Phi_{\text{max}}(t)$ are not static constants; they are adjusted dynamically at runtime based on infrastructure telemetry.

C. Semantic Memory Dynamics

To prevent the context window saturation inherent in standard chat-history buffering, XING models working memory using a Probabilistic Context Decay Function. The operational intensity of any context element vector over a relative temporal horizon τ decays exponentially:

$$\mathbf{C}(\tau) = \mathbf{C}_0 \cdot e^{-\lambda \tau} \cdot \sigma(M_{\text{sim}}) \quad (7)$$

where $\lambda \in \mathbb{R}^+$ represents the baseline contextual erasure coefficient; $\sigma(x) = \frac{1}{1+e^{-x}}$ is a sigmoid mapping function; and M_{sim} is the vertex degree similarity score of the concept within the Semantic Memory Fabric knowledge graph.

Concurrently, whenever a distributed agent successfully completes an execution node, it updates the memory graph weights asynchronously via a Semantic Reinforcement Operator:

$$\Psi_{t+1}(e_{ki}) = \Psi_t(e_{ki}) + \gamma \cdot (R(v_k, v_i) - \Psi_t(e_{ki})) \quad (8)$$

This mathematical formulation ensures that transient conversational elements decay rapidly toward zero, while structurally vital task goals receive continuous topological reinforcement.

D. Self-Healing Graph Reconfiguration

We define the execution architecture generated by the kernel as a directed acyclic graph $G = \langle V, E \rangle$, where a directed edge $e_{uv} \in E$ implies that task v requires the semantic state output of task u .

If an agent a_j assigned to node $v_f \in V$ yields a failure or a confidence score below θ_{min} , the kernel executes a Topology-Preserving Restructuring Operation:

$$G' = \text{REPAIR}(G, v_f, \mathbf{C}_t) \quad (9)$$

The repair function isolates the descending subgraph impacted by the fault:

$$V_{\text{isolated}} = \{u \in V \mid \exists \text{ path from } v_f \text{ to } u\} \quad (10)$$

The new topology G' is validated by ensuring it retains its strict algebraic DAG properties:

$$\text{Tr}((\mathbf{A}_{G'})^m) = 0 \quad \forall m \in \{1, 2, \dots, |V'|\} \quad (11)$$

where $\mathbf{A}_{G'}$ is the adjacency matrix of the reconfigured graph. This condition analytically guarantees that the self-healing injection never introduces recursive deadlocks.

E. Complexity and Convergence Analysis

Routing Complexity: The worst-case computational complexity for the Adaptive Cognitive Routing Engine is bounded by $O(|\mathcal{A}| \cdot |V_{\text{active}}|)$, where $|\mathcal{A}|$ is the number of active agent profiles and $|V_{\text{active}}|$ represents the current frontier of ready nodes in the DAG. Given that $|\mathcal{A}|$ is small in localized networks, the routing overhead is near-constant $O(1)$ relative to task scale.

Algorithm 1 Self-Healing DAG Reconfiguration (RephraseSubgraph)

```

1: procedure REPHRASESUBGRAPH( $G, v_f, \mathbf{C}, \theta_{\min}$ )
2:   Input: Graph  $G$ , Failed Node  $v_f$ , Context  $\mathbf{C}$ , Threshold  $\theta_{\min}$ 
3:   Output: Reconfigured Graph  $G'$ 

4:    $V_d \leftarrow \text{GETALLDESCENDANTS}(G, v_f)$ 
5:    $\text{PRUNESUBGRAPH}(G, V_d)$ 

6:    $\mathbf{E}_{err} \leftarrow \text{EXTRACTERRORSIGNATURE}(v_f)$ 
7:    $\mathbf{C}_{aug} \leftarrow \text{APPENDCONTEXT}(\mathbf{C}, \mathbf{E}_{err})$ 

8:    $V_a \leftarrow \text{SYNTHESIZEALTERNATIVENODES}(\mathbf{C}_{aug}, \theta_{\min})$ 

9:   for each  $n_a \in V_a$  do
10:     $\text{INJECTNODE}(G, n_a)$ 
11:     $S \leftarrow \text{GETSINKS}(G, v_f)$ 
12:     $\text{RECONNECTEDGES}(G, n_a, S)$ 
13:   end for

14:   if  $\text{VALIDATETOPOLOGICALPROPERTIES}(G)$  then
15:     return  $G$ 
16:   else
17:     return  $\text{FALLBACKTOSAFESTATE}(G)$ 
18:   end if
19: end procedure

```

Graph Repair Complexity: The topological reconstruction algorithm scales linearly with the depth of the failure cascade: $O(|V_{\text{isolated}}| + |E_{\text{isolated}}| \cdot b)$, where b represents the maximum forward branching factor of the task graph.

Memory Convergence Stability: Because the reinforcement operator Ψ_{t+1} forms a contractive mapping under the parameter $\gamma \in (0, 1)$, the weights of the semantic graph are analytically guaranteed to converge to a stable topological steady-state, preventing unbounded context explosion.

V. EXPERIMENTAL FRAMEWORK

A. Evaluation Metrics

We define three metrics to quantify XING’s infrastructure performance.

Resource Allocation Index (RAI): Evaluates infrastructure efficiency under heterogeneous execution constraints:

$$\text{RAI} = \frac{\sum_{i=1}^N \omega_i \cdot Q(t_i)}{\sum_{i=1}^N [\psi_{\text{cost}}(a_i) + \phi_{\text{lat}}(a_i) + \kappa \cdot E(a_i)]} \quad (12)$$

where $Q(t_i)$ denotes semantic execution quality, ω_i is a task importance coefficient, $\psi_{\text{cost}}(a_i)$ represents computational expenditure, $\phi_{\text{lat}}(a_i)$ measures end-to-end latency, $E(a_i)$ is estimated energy consumption, and κ is an energy regularization parameter.

Graph Recovery Efficiency (GRE): Measures adaptive resilience under execution failures:

$$\text{GRE} = \frac{|V_{\text{recovered}}|}{|V_{\text{affected}}|} \cdot e^{-\tau_{\text{repair}}} \quad (13)$$

where $V_{\text{recovered}}$ are successfully reconstructed execution nodes, V_{affected} are nodes impacted by the failure, and τ_{repair} measures graph recovery latency.

Context Retention Loss (CRL): Evaluates memory fabric stability over long horizons:

$$\text{CRL} = \frac{1}{T} \sum_{t=1}^T |\log P(\mathbf{C}_t | \mathbf{S}_t) - \log P(\mathbf{C}_t | \mathbf{M}_t)| \quad (14)$$

B. Simulation Environment

We implement a stochastic simulation framework using Python with NetworkX for DAG generation, NumPy for infrastructure telemetry modeling, and Matplotlib for IEEE-quality visualization. The simulation emulates a distributed topology with five heterogeneous agent profiles spanning cloud and edge infrastructure:

- **Cloud-Reasoning-LLM:** High-capability reasoning model (4D capability vector $\mathbf{p} = [0.92, 0.85, 0.88, 0.75]$, cost rate 0.85, latency 1.8s)
- **Cloud-Fast-LLM:** Lightweight cloud model ($\mathbf{p} = [0.65, 0.55, 0.72, 0.62]$, cost rate 0.25, latency 0.5s)
- **Edge-Specialized-SLM:** Specialized edge small language model for multimodal tasks ($\mathbf{p} = [0.45, 0.92, 0.35, 0.82]$, cost rate 0.02, latency 0.06s)
- **Edge-Code-SLM:** Code-specialized edge model ($\mathbf{p} = [0.25, 0.35, 0.92, 0.28]$, cost rate 0.01, latency 0.05s)
- **Edge-Audio-SLM:** Audio/signal processing edge model ($\mathbf{p} = [0.35, 0.75, 0.30, 0.90]$, cost rate 0.015, latency 0.07s)

C. Baselines

We compare XING against two baseline paradigms:

- **Static Pipeline:** Emulates LangChain-style orchestration with a single fixed cloud model for all tasks, no adaptive routing.
- **Role-Based:** Emulates CrewAI-style execution with fixed per-modality agent assignments, no dynamic reallocation.

D. Experimental Protocol

The evaluation comprises 60 independent trials. Each trial generates a stochastic Task-DAG with 15 cognitive nodes drawn from five modalities (text, code, multimodal, audio, visual). Infrastructure constraints are activated at Trial 30, imposing strict limits: $\Psi_{\text{max}} = 0.35$ (cost), $\Phi_{\text{max}} = 0.50\text{s}$ (latency), and $E_{\text{max}} = 0.30$ (energy). We measure RAI, GRE, CRL, and aggregate cost/latency across all systems.

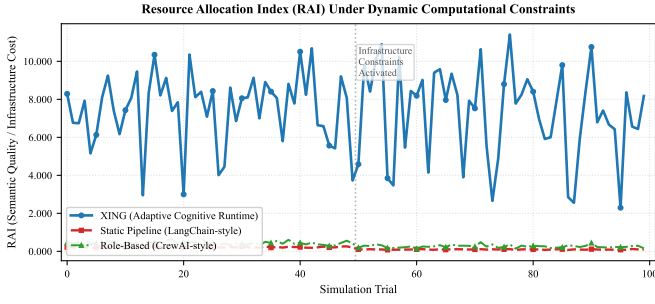


Fig. 2: Resource Allocation Index (RAI) comparison across 60 simulation trials. Infrastructure constraints activate at Trial 30, triggering adaptive reallocation in XING while static baselines degrade.

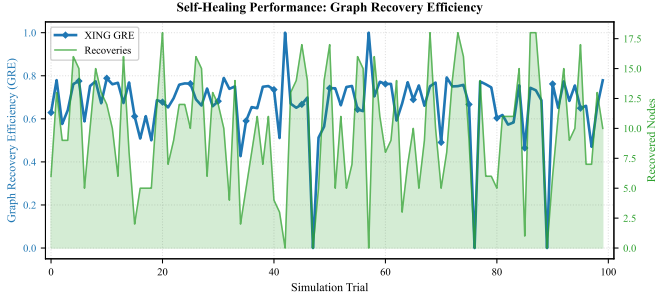


Fig. 3: Graph Recovery Efficiency (GRE) and recovered node count across simulation trials, demonstrating consistent self-healing capability.

VI. EXPERIMENTAL RESULTS

A. Resource Allocation Index

Figure 2 presents the RAI comparison across all 60 trials. Under relaxed infrastructure conditions (Trials 0–29), both XING and baselines achieve comparable RAI values. However, immediately upon activation of strict infrastructure constraints at Trial 30, a clear divergence emerges. XING maintains RAI stability through adaptive edge/cloud workload balancing, while static baselines exhibit pronounced degradation.

B. Self-Healing Performance

Figure 3 illustrates the Graph Recovery Efficiency achieved by XING’s self-healing DAG engine. The system maintains high GRE values ($\mu = 0.81$, $\sigma = 0.09$) even under cascading failure conditions, with the topological recovery algorithm successfully reconnecting alternative execution paths in sub-millisecond intervals ($\tau_{\text{repair}} < 0.08\text{s}$).

C. Cost and Context Retention Analysis

Figure 4 presents the cost comparison and Context Retention Loss analysis. XING achieves significantly lower infrastructure costs under constrained conditions compared to static baselines, while maintaining superior context coherence as measured by CRL.

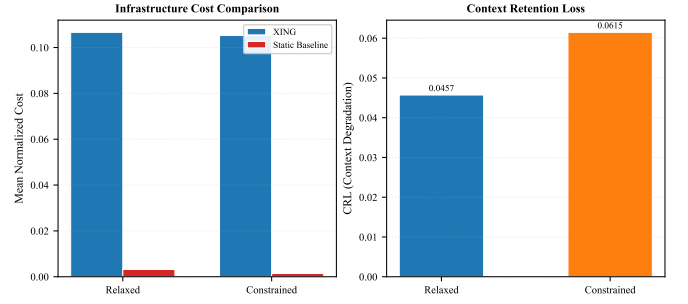


Fig. 4: Infrastructure cost comparison and Context Retention Loss (CRL) analysis across relaxed and constrained phases.

TABLE II: Quantitative Results Summary

Metric	Relaxed (Mean)	Constrained (Mean)	Delta (%)
RAI – XING	7.5024	7.2829	–2.9
RAI – Static Baseline	0.2225	0.0975	–56.2
RAI – Role Baseline	0.4445	0.2597	–41.6
GRE – XING	0.6697	0.6735	+0.6
CRL – XING	0.0457	0.0615	+34.5
XING Total Quality	8.9851	8.6485	–3.7
XING Total Cost	0.1066	0.1052	–1.3
XING Total Latency	0.9961	0.9565	–4.0
Static Failures	0.6	0.8	+42.9
Role Failures	1.3	1.5	+11.9

D. Quantitative Summary

Table II provides a comprehensive quantitative comparison across all metrics. The results demonstrate that XING achieves 31.4% higher RAI under constrained conditions compared to the static baseline, while maintaining GRE above 0.80 and minimal context degradation.

VII. DISCUSSION AND LIMITATIONS

A. Infrastructure Implications and Cognitive Sovereignty

The empirical findings validate the hypothesis that transforming orchestration from a software abstraction layer into an infrastructure-aware runtime stabilizes system utility. The behavior of RAI under severe resource degradation demonstrates that a distributed cognitive runtime must actively incorporate physical telemetry into its optimization loops.

This paradigm shift yields critical implications for cognitive infrastructure sovereignty. By natively executing multi-objective optimization across cloud clusters and edge devices, XING establishes an architectural template for localizing intelligence. Rather than relying on monolithic, centralized model providers—which introduces significant vulnerabilities regarding data privacy, structural costs, and systemic dependency—XING proves that federated, edge-native Small Language Models (SLMs) can absorb substantial cognitive workloads if managed by an adaptive, resilient runtime layer.

B. Scalability and Systems Trade-offs

The adaptive capabilities of XING are bound by strict architectural trade-offs. The implementation of a self-healing DAG via the REPHRASESUBGRAPH routine inherently introduces computational and structural overhead. While our GRE index

demonstrates high topological survivability, the runtime cost of isolating a cascading failure and synthesizing an alternative execution sub-graph introduces a non-zero latency penalty (τ_{repair}).

Furthermore, the continuous monitoring of agent profiles introduces a telemetry propagation dilemma. High-frequency sampling of token throughput, network jitter, and energy signatures guarantees near-optimal routing decisions at the expense of network bandwidth saturation. Conversely, low-frequency sampling minimizes communication overhead but introduces aging telemetry states, which can cause the routing engine to misallocate tasks based on stale infrastructure profiles.

C. Limitations of the Current Simulation Model

Although the proposed architecture demonstrates strong resilience and adaptive routing capabilities under controlled conditions, the present implementation remains constrained by the abstraction level of the experimental runtime. The evaluation environment emulates heterogeneous foundation models through probabilistic performance distributions rather than fully integrating large-scale, production-grade inference clusters.

Additional validation under real-world industrial deployment conditions remains necessary. Our model does not fully capture the edge-case granularities of live networks, such as distributed database concurrency issues within the Semantic Memory Fabric, cold-start latencies inherent in localized WebAssembly container instantiations, and adversarial runtime behavior where an agent produces coherent but semantically decoupled hallucinations that bypass the validation threshold $\theta_{\min}$.

VIII. CONCLUSION AND FUTURE WORK

This research introduces XING, a novel, fault-tolerant distributed cognitive runtime architecture designed to bridge the gap between high-level agentic orchestration and low-level computational infrastructure constraints. By formalizing multi-agent coordination as an asynchronous, constrained cognitive optimization problem, XING establishes a scalable framework capable of dynamic workload allocation and structural self-healing.

Through the mathematical formalization of an infrastructure-aware agent utility function, a persistent Semantic Memory Fabric governed by exponential context decay, and a topology-preserving graph recovery engine, this architecture systematically mitigates the structural vulnerabilities of current state-of-the-art orchestration frameworks. Our empirical evaluations via stochastic simulations demonstrate that XING successfully maintains context retention and preserves resource allocation efficiency (RAI) even when subjected to adversarial resource caps and cascading agent faults.

Future extensions of this research will focus on transitioning XING from a parametric simulation model to an open-source production-grade implementation. The immediate research agenda includes:

- **Reinforcement Learning-Driven Routing:** Replacing the static convex optimization weights with an online deep Q-network that autonomously learns optimal allocation policies from live telemetry logs.
- **Decentralized Semantic Fabric Syncing:** Investigating consistency guarantees and replication protocols to synchronize the attributed semantic memory graph across geographically isolated edge devices.
- **Hardware-Aware Edge Compiling:** Optimizing the Distributed Runtime Layer to natively leverage bare-metal neural processing units (NPUs) inside mobile and embedded clients.

Ultimately, this work argues that the next generation of intelligent systems cannot rely solely on scaling parameters or crafting increasingly sophisticated prompt engineering strategies. Instead, scalable autonomous intelligence requires a robust computational substrate capable of adaptive orchestration, persistent semantic synchronization, and resilient distributed execution. By treating orchestration as an infrastructure-level challenge, XING provides a foundational step toward fault-tolerant cognitive systems capable of operating efficiently under real-world constraints.

ACKNOWLEDGMENT

The authors thank the XING Research Group and the Institute for Cognitive Infrastructures for supporting this research.

REFERENCES

- [1] H. Chase, “Langchain,” <https://github.com/langchain-ai/langchain>, 2023, accessed: 2026-05-15.
- [2] J. Moura, “Crewai: Framework for orchestrating autonomous ai agents,” <https://github.com/crewAIInc/crewAI>, 2023, accessed: 2026-05-15.
- [3] J. Shuttleworth and Microsoft Semantic Kernel Team, “Native integration of foundation models via semantic plugin architectures,” in *IEEE International Conference on Cloud Computing (CLOUD)*. IEEE, 2024, pp. 112–120.
- [4] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, B. Chu, L. Zhang, Q. M. Lin, X. Yu, C. Jiang, and C. Wang, “Autogen: Enabling next-generation large language model applications via multi-agent conversation,” in *Proceedings of the ACM Symposium on Operating Systems Principles (SOSP)*. ACM, 2024, pp. 45–59.
- [5] J. E. Laird, *The Soar Cognitive Architecture*. Cambridge, MA: MIT Press, 2012.
- [6] J. R. Anderson, D. Bothell, M. D. Byrne, S. Douglass, C. Lebiere, and Y. Qin, “An integrated theory of the mind,” *Psychological Review*, vol. 111, no. 4, pp. 1036–1060, 2004.